

Section: CS

May 20, 2026

Multi-Signature Authentication Web Proxy

Ian Barish

Problem Statement

Enterprise grade cryptocurrency wallets have long implemented multi-signature (“multi-sig”) authentication, which requires two or more parties to sign an outbound transaction from a secure wallet before it is sent to the receiving party. This security mechanism ensures that before a transaction is sent, it is agreed upon by the group, rather than any one individual. But what if you want to require multiple parties to approve of a critical setting change or a login to a sensitive online account? No existing general-purpose web authentication system (think Duo or Authelia) requires multiple distinct people to cooperate before granting access.

My practicum project proposes a general-purpose multi-signature authentication web proxy that allows access to the requested resource only when multiple parties provide their secret keys.

Motivation

This may be used by organizations who want to ensure product updates are only sent to end users when approved by multiple parties, preventing any one developer from owning the “keys to the kingdom.” This could prevent the supply-chain attacks we have been seeing wreak havoc on npm and PyPi. There is also a use case for individuals. For example, separated parents who self-manage a savings account containing the child’s college fund. The parents may want a way to ensure both parties consent when one of the parents attempts to access the child’s college fund.

Expected Deliverables

When the project is complete, I will have an open-source application runnable in a docker container that hosts a web server which acts as an authentication web proxy similar to Duo or Authelia. When a user requests access to a resource, the web proxy catches the request, checks its access control list (ACL), and, if multi-sig authentication is required for that resource, notifies the other “approvers” to come and approve the request. When a signer attempts to approve a request, they must authenticate into the web proxy or otherwise present their secret key (to be decided based on research). Once enough approvers approve the request (a “quorum”), which can be configured in the ACL, the original user to is allowed to authenticate into the system. The system should be able to handle arbitrary, applications whether they are internal or external without needing to modify the end application. For internal applications, the proxy can sit directly in front of the end application like a traditional authentication proxy. For external applications, the login flow gets complicated as a user could bypass the system and enter their credentials directly on the

website. This is a problem I intend to research and solve.

Once the basic functionality of the application has been created (as defined above), I intend to explore additional functionality to add to the design. For example, while it is helpful to gate access behind multiple approvals, it would be best if those approvers could then see the user's screen while they are navigating the application. As an audit afterward, the application could save the screen share to a video file that can be reviewed later. I would also like to ensure that my project works with popular web application proxies like Traefik, which I use personally.